Sample Report

Author McWriterson

Winter 2024/2025

Spis treści

1 Zadanie 1 **1**
1.1 Polecenie: . . . 1
1.2 Pytania: . . . 2
1.3 Wyniki: . . . 2
1.4 Wnioski: . . . 2

2 Zadanie 2 **2**
2.1 Polecenie: . . . 2
2.2 Pytania: . . . 3
2.3 Wyniki: . . . 3
2.4 Wnioski: . . . 3

3 Zadanie 3 **3**
3.1 Polecenie: . . . 3
3.2 Pytania: . . . 3
3.3 Wyniki: . . . 3
3.4 Wnioski: . . . 3

4 Zadanie 4 **4**
4.1 Metody iteracyjne . . . 4
4.2 Polecenie . . . 4
4.2.1 Metoda Jacobiego . . . 4
4.2.2 Metoda Gaussa-Seidela: . . . 4
4.2.3 Metoda SOR: . . . 4
4.3 Wyniki i wnioski: . . . 4
4.3.1 parametr k . . . 5
4.3.2 Parametr c . . . 5
4.4 Propozycje . . . 6

1 Zadanie 1

1.1 Polecenie:

Dane: x, e - wektory kolumnowe należące do $\mathbb{R}^n$.

$$k = \frac{\|x\|}{\|e\|} = \sqrt{\frac{x^T x}{e^T e}}, \quad v = x - ke, \quad u = x + ke.$$

Napisz funkcję

```
function [P, Q] = haus(x, e)
```

która zwraca macierze P oraz Q dane wzorami:

$$P = I - 2\frac{vv^T}{v^T v}, \quad Q = I - 2\frac{uu^T}{u^T u}.$$

Wywołanie funkcji:

```
x = [1; 0; 0; 0];
e = [1/2; -1/2; -1/2; -1/2];
[P, Q] = haus(x, e)
```

< z slajdów >

1.2 Pytania:

1.3 Wyniki:

n	k	Błąd 1	Błąd 2	Błąd 3	Błąd 4
3	1.0892	$2.2204e-16$	$2.2204e-16$	$2.2204e-16$	$2.2204e-16$
3000	0.9829	$1.7764e-15$	$1.9984e-15$	$3.2198e-15$	$2.9976e-15$
3000; e prawie równy x	1.000	$1.9984e-15$	$1.8874e-15$	$2.6645e-15$	$3.8858e-15$
3000; x, e ortogonalnych:	0.9939	$1.9984e-15$	$1.7764e-15$	$4.2188e-15$	$3.5527e-15$

1.4 Wnioski:

2 Zadanie 2

2.1 Polecenie:

Dane: $B := A$ - macierz nieosobliwa.

Algorytm QR:

for $m := 1$ **to** n **do**

$t := \sum_{s=m}^{n}(B[s,m])^2$;

if $B[m,m] < 0$ **then** $k := \sqrt{t}$ **else** $k := -\sqrt{t}$;

$d[m] := k$;

$f[m] := \frac{1}{k \cdot B[m,m] - t}$;

$B[m,m] := B[m,m] - k$;

for $i := m+1$ **to** n **do**

$y := f[m] \sum_{s=m}^{n} B[s,m]B[s,i]$;

for $j := m$ **to** n **do** $B[j,i] := B[j,i] + B[j,m] \cdot y$;

Wynik:

$$R = \begin{bmatrix} d[1] & B[1,2] & \cdots & B[1,n] & \\ 0 & d[2] & B[2,3] & \cdots & B[2,n] \\ \vdots & \ddots & \ddots & \dots & \\ 0 & \cdots & d[n-1] & B[n-1,n] & \\ 0 & \cdots & 0 & d[n] & \end{bmatrix}, \quad Q = AR^{-1}.$$

Napisz funkcję:

```
function [Q, R] = QR(A)
```

zwracającą macierze Q i R, a następnie znajdź rozkład QR dla wylosowanych macierzy.

2.2 Pytania:

2.3 Wyniki:

n	błąd-1	błąd-2	błąd-3	błąd-4
3	$8.8818e-16$	$5.5511e-17$	$4.4409e-16$	$3.3307e-16$
500	$5.4368e-13$	$3.5416e-14$	$1.1102e-15$	$7.9381e-15$
100	$3.9968e-15$	$5.6843e-14$	$1.7764e-15$	$3.1264e-13$

2.4 Wnioski:

3 Zadanie 3

3.1 Polecenie:

Dla macierzy

$$A = \begin{bmatrix} a_{1,1} & a_{1,2} & \cdots & a_{1,n} \\ a_{2,1} & a_{2,2} & \cdots & a_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n,1} & a_{n,2} & \cdots & a_{n,n} \end{bmatrix}$$

Norma druga macierzy A to maksimum po pierwiastkach wielomianu charakterystycznego macierzy A^TA, czyli:

$$w(x) := \det(A^TA - xI), \quad C := \{x : w(x) = 0\}, \quad \|A\|_2 = \max_{x \in C} \sqrt{x}.$$

Napisz funkcję

```
ret = norm2(A)
```

która zwraca normę drugą macierzy A.

3.2 Pytania:

3.3 Wyniki:

3.4 Wnioski:

Porównując naszą funkcję *norm*2 z wbudowaną w Octave funkcją *norm* otrzymujemy rozbieżności wynikające z błędów numerycznych. W naszej funkcji błędy te są spowodowane głównie obliczaniem pierwiastków wielomianu charakterystycznego naszej macierzy. Jest to szczególnie wrażliwa operacja dla macierzy o bliskich sobie wartościach własnych. Zespolone pierwiastków nie powinny się pojawiać,

jednak dla macierzy płaskich błędy mogą być liczbami zespolonymi ze względu na możliwą małą część zespoloną pierwiastków wielomianu, ale także na błędy zaokrągleń przy obliczaniu wielomianu charakterystycznego, które później mogą skutkować pojawieniem się niepożądanych wartości zespolonych pierwiastków przy dużych macierzach. Dla macierzy z dużymi wartościami na przekątnej może dojść do błędnych zaokrągleń spowodowanych przepełnieniem pamięci w przypadku ekstremalnych wartości. Aby wyzbyć się błędów spowodowanych liczeniem pierwiastków, można użyć np. metody potęgowej. Koncentruje się ona wyłącznie na największej wartości własnej, która w miarę iteracji staje się bardziej widoczna oraz wymaga tylko mnożenia macierzy przez wektor, co zmniejsza ryzyko występowania błędów.

4 Zadanie 4

4.1 Metody iteracyjne

4.2 Polecenie

4.2.1 Metoda Jacobiego

Dane: A - macierz $n \times n$, b, x_0 - wektory kolumnowe należące do $\mathbb{R}^n$, k - liczba naturalna.
Niech $B := D$.
Napisz funkcję

```
function retval = Jacobi(A, b, x0, k)
```

która zwraca wektor $x^{(k)}$.

4.2.2 Metoda Gaussa-Seidela:

Dane: A - macierz $n \times n$, b, x_0 - wektory kolumnowe należące do $\mathbb{R}^n$, k - liczba naturalna.
Niech $B := D + L$.
Napisz funkcję

```
function retval = GaussSiedel(A, b, x0, k)
```

która zwraca wektor $x^{(k)}$.

4.2.3 Metoda SOR:

Dane: A - macierz $n \times n$, b, x_0 - wektory kolumnowe należące do $\mathbb{R}^n$, k - liczba naturalna, $t \in (0, 2)$.
Niech $B := \frac{1}{t}D + L$.
Napisz funkcję

```
function retval = SOR(A, b, x0, k, t)
```

która zwraca wektor $x^{(k)}$.

4.3 Wyniki i wnioski:

Przy wszystkich wyliczeniach ustalamy $x = 0, b = rand(n, 1) * 10$, robimy 1000 prób i bierzemy z nich średnią.

4.3.1 parametr k

Szacujemy k, czyli liczbę iteracji potrzebnych do stwierdzenia zbieżności macierzy. Aby uniknąć błędów związanych z brakiem dominacji diagonalnej, ustalamy $c = n \cdot 10$. Dzięki temu zapewniamy dominującą przekątną w macierzy, co eliminuje błędy wynikające z jej struktury. W takiej sytuacji możemy ustalić $n = 10$, ponieważ przy tej wielkości macierzy nie będą pojawiały się problemy związane z jej rozmiarem i brakiem zbieżności.

Metoda	1	3	5	10	16
Jacobi	4.0556e-03	6.5079e-06	9.9949e-09	2.7341e-15	1.1819e-17
Gauss-Siedel	3.3702e-03	4.4534e-07	4.7716e-12	2.6600e-17	1.8228e-17
SOR	0.010987	1.3029e-04	1.2422e-06	2.5774e-11	2.3522e-17

Wnioski:

- Metoda SOR – wpływ źle dobranego współczynnika relaksacji:
  Metoda SOR, choć teoretycznie powinna charakteryzować się najszybszą zbieżnością, w naszych obliczeniach tego nie potwierdza. Wynika to z faktu, że zastosowano stały współczynnik relaksacji, który został nieodpowiednio dobrany. W efekcie metoda SOR wykazuje słabszą efektywność. Można zatem stwierdzić, że przy źle dobranym współczynniku relaksacji skuteczność tej metody znacząco maleje.

- Porównanie metod Jacobiego i Gaussa-Seidla:
  Analiza wyników pokazuje, że metoda Gaussa-Seidla osiąga szybszą zbieżność już od pierwszej iteracji. Wynika to z charakteru algorytmu, w którym nowe wartości obliczane w bieżącej iteracji są natychmiast wykorzystywane w kolejnych obliczeniach. Z kolei metoda Jacobiego bazuje wyłącznie na wartościach z poprzedniej iteracji, co spowalnia proces zbieżności.

4.3.2 Parametr c

Szacujemy c, które jest parametrem odpowiedzialnym za wartość dodaną na diagonali w macierzy $A + c \cdot \text{eye}(n)$. Ustalamy $k = 100$, aby mieć pewność co do zbieżności.

- przypadek dla $c = n/2$

Metoda	3	10	100
Jacobi	$1.4872e-08$	$2.0980e-03$	$8.8618e-03$
Gauss-Siedel	$5.8098e-16$	$3.9518e-16$	$17138e-16$
SOR	$5.9233e-16$	$3.9828e-16$	$17124e-16$

- przypadek dla $c = 10 * n$

Metoda	3	10	100
Jacobi	$3.9344e-17$	$2.5617e-17$	$1.0956e-17$
Gauss-Siedel	$4.0613e-17$	$2.5840e-17$	$1.0976e-17$
SOR	$3.7866e-17$	$2.5573e-17$	$1.0902e-17$

Wnioski:
Możemy zauważyć że takie przybliżenie c które sugeruje nam średnia $E[\sum_i^n X_i]$, gdzie $X_1 \sim \text{U}([0,1])$ jest dobrym szacowaniem. Błędy związane z metodą Jacobiego musza wynikac z samego algorytmu, ale nii rozumiem dlaczego pojawiają się dla

4.4 Propozycje

Moment stopu:
Ustalamy p i definiujemy moment stopu w momencie n jeśli pradziwe będzie wyrażenie:

$$\left|\frac{x_n - x_{n-1}}{x_n}\right| \times 100 < p$$